

# OpenZeppelin Stack for Midnight: Compact Contracts Audit



**May 20, 2026**

*This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.*

Copyright © 2026 Zeppelin Group Ltd.

# Table of Contents

|                                                                              |          |
|------------------------------------------------------------------------------|----------|
| Table of Contents                                                            | 2        |
| Summary                                                                      | 3        |
| Scope                                                                        | 4        |
| System Overview                                                              | 5        |
| Security Model and Trust Assumptions                                         | 5        |
| Extensibility Pattern and the Safe/Unsafe Circuit Distinction                | 5        |
| Witness Privacy                                                              | 6        |
| Contract-to-Contract Calls and the Unsafe Circuit Pattern                    | 6        |
| Module Composition                                                           | 7        |
| Privileged Roles                                                             | 7        |
| <b>Critical Severity</b>                                                     | <b>8</b> |
| C-01 Access Control Can Be Bypassed Through Arbitrary ownPublicKey Witnesses | 8        |
| <b>Low Severity</b>                                                          | <b>9</b> |
| L-01 Zero-value transfers may revert outside an assertion                    | 9        |
| L-02 Non-Existent and Zero Allowances Have Inconsistent Behavior             | 9        |
| L-03 Token URIs Persist Across Burn and Remint                               | 10       |
| L-04 Misleading Documentation Regarding DEFAULT_ADMIN_ROLE                   | 10       |
| L-05 Owner Can Be Zero                                                       | 11       |
| L-06 Code Simplifications                                                    | 11       |
| L-07 Missing Admin Role Holder                                               | 12       |
| L-08 Misleading Comments                                                     | 12       |
| Notes & Additional Information                                               | 13       |
| N-01 Typographical Errors                                                    | 13       |
| Conclusion                                                                   | 14       |
| Appendix                                                                     | 15       |
| Issue Classification                                                         | 15       |

# Summary

|           |                                  |                                |                 |
|-----------|----------------------------------|--------------------------------|-----------------|
| Type      | Library                          | Total Issues                   | 10 (8 resolved) |
| Timeline  | From 2026-03-16<br>To 2026-04-07 | Critical Severity Issues       | 1 (1 resolved)  |
| Languages | Compact                          | High Severity Issues           | 0 (0 resolved)  |
|           |                                  | Medium Severity Issues         | 0 (0 resolved)  |
|           |                                  | Low Severity Issues            | 8 (6 resolved)  |
|           |                                  | Notes & Additional Information | 1 (1 resolved)  |
|           |                                  | Client Reported Issues         | 0 (0 resolved)  |

# Scope

OpenZeppelin performed an audit of

- the [OpenZeppelin/compact-contracts](#) repository at the [d8ee045](#) commit.
- the [OpenZeppelin/midnight-apps](#) repository at the [7e09763](#) commit.
- the [OpenZeppelin/compact-contracts](#) repository's [pull request #412](#) at the [a7a9aa4](#) commit.

In scope were the following files:

```
compact-contracts/contract/src
├── access
│   ├── AccessControl.compact
│   ├── Ownable.compact
│   ├── ShieldedAccessControl.compact
│   └── ZOwnablePK.compact
├── security
│   ├── Initializable.compact
│   └── Pausable.compact
├── token
│   ├── FungibleToken.compact
│   ├── MultiToken.compact
│   └── NonFungibleToken.compact
└── utils
    └── Utils.compact

midnight-apps/contracts/src/math
├── Bytes32.compact
├── Bytes8.compact
├── Field255.compact
├── Pack.compact
├── Types.compact
├── Uint128.compact
├── Uint256.compact
└── Uint64.compact
```

**Update:** The fixes for the findings highlighted in this report have all been merged at commit [9b6b21a](#).

# System Overview

OpenZeppelin Contracts for Compact is an open-source library of reusable modules for secure smart contract development on the [Midnight](#) network, which is a privacy-focused blockchain that uses zero-knowledge proofs to protect private state. The contracts are written in [Compact](#), a domain-specific language that compiles down to ZK circuits. Each circuit execution produces a cryptographic proof that a state transition was computed correctly, without necessarily revealing the private inputs used in that computation.

The library covers five areas: access control, shielded (privacy-preserving) access control, token standards, security utilities, and math operations (currently in [midnight-apps repo](#)). The library modules are not deployed independently: they are imported into a top-level contract that composes them and selectively exposes circuits to users.

## Security Model and Trust Assumptions

### Extensibility Pattern and the Safe/Unsafe Circuit Distinction

The library follows a modular composition model described in the [OpenZeppelin extensibility documentation](#). Modules expose two categories of circuits for external use:

- **External circuits** are standalone operations designed to be re-exported directly from an implementing contract without modification. They carry their own access control and input validation, so exposing them as-is is safe. Examples include `transfer` in `FungibleToken` and `assertOnlyOwner` in `Ownable`.
- **Public circuits** are composable building blocks that may be missing access control or other security checks so that implementing contracts can combine them into custom flows. They are identified by an underscore prefix, such as `_mint`, `_burn`, `_grantRole`, and `_transferOwnership`. This prefix signals that the circuit is **not**

safe to expose directly at the implementing contract's external surface. Note that this is just a convention and is not enforced at compile-time.

## Witness Privacy

Circuit input values can be provided directly (as circuit arguments) or using the *witness function* mechanism, where an externally-defined function - local to the executing node - computes or retrieves the value, potentially using the current state of the circuit execution. Either way, the inputs remain private by default and are never revealed to verifiers.

However, any values that are read from or written to the contract's state (including mapping keys) will be publicly visible in the transaction transcript. To ensure this is explicitly acknowledged, private values that are revealed in this way must be enclosed in a `disclose` annotation.

This audit assumes that the ZK proof system is sound, meaning that accepted state transitions must satisfy all circuit constraints, input values are known to the prover, and the transcript does not reveal anything about private values that are not explicitly disclosed.

## Contract-to-Contract Calls and the Unsafe Circuit Pattern

Compact does not currently support contract-to-contract (C2C) calls. This has a direct consequence for access control: a contract cannot pass any authentication check that is evaluated by another contract.

The library acknowledges this limitation through an explicit naming convention. As a safety precaution, the standard functions will refuse to assign ownership, tokens and roles to contract addresses. However, there are functions prefixed with `_unsafe` (such as `_unsafeGrantRole` in `AccessControl`) that developers can use to bypass this restriction.

Once C2C calls are enabled, the standard functions will support contract addresses (i.e. they will behave like their unsafe counterparts) and the unsafe versions will be deprecated. Until then, any tokens or privileges granted to a `ContractAddress` are unusable.

# Module Composition

Compact does not support inheritance. Instead, the library uses explicit delegation: a top-level contract imports modules under a namespace prefix and calls their circuits directly. There is no automatic state wiring, constructor chaining, or method resolution order. Each module owns its own ledger state, circuits, types, and witness declarations, all namespaced under the import prefix to prevent collisions.

This means module initialization is never automatic. A module that depends on `Initializable` will silently operate on zero-default state if the implementing contract omits the corresponding `initialize` call. Unlike a Solidity constructor that runs whenever a descendant contract is deployed, a Compact module's setup circuit must be invoked deliberately, in the correct order, from the top-level constructor.

# Privileged Roles

The specific privileged roles vary by module, but the general trust model is consistent across the library: accounts holding administrative roles are assumed to be honest. The library provides no on-chain mechanism to limit, time-lock, or reverse the actions of a legitimately credentialed admin. Implementing contracts are responsible for establishing governance processes that reduce the risk of admin key compromise or misuse.

# Critical Severity

## C-01 Access Control Can Be Bypassed Through Arbitrary `ownPublicKey` Witnesses

The `AccessControl`, `Ownable`, `FungibleToken`, `MultiToken`, and `NonFungibleToken` contracts assume that `ownPublicKey` identifies the transaction constructor and can therefore be used for authorization checks.

However, `ownPublicKey` is [a witness that can be set arbitrarily](#). The transaction lifecycle does not require the caller to prove control of the corresponding private key. As a result, these checks do not authenticate the caller. Any user can supply a chosen `ownPublicKey` value and satisfy logic that is intended to restrict access based on ownership, roles, or token permissions. This breaks the security assumptions of the unshielded library and can allow unauthorized execution of protected operations, including restricted token transfers.

Consider replacing `ownPublicKey` based identity checks with an identity derived from a secret witness, such as a hash of that secret, so authorization depends on knowledge that cannot be chosen arbitrarily while preserving the intended persistent identity properties, including linkability across actions.

**Update:** Resolved in pull request [#444](#), [#466](#), [#468](#), [#469](#), and [#483](#).

*The following remediations were implemented:*

1. **Scope witness names:** prevent forced key reuse when composing modules in the same contract e.g. `witness wit_FungibleTokenSK(): Bytes<32>;`
2. Replace `ownPublicKey` with `H(secretKey)`
3. Change `Either<ZswapCoinPublicKey, ContractAddress>` to `Either<Bytes<32>, ContractAddress>`

# Low Severity

## L-01 Zero-value transfers may revert outside an assertion

The `_update` function in the `MultiToken` contract [reduces the source account's balance](#) without checking if the token `id` record has been initialized. In practice, this will only occur when the `value` is set to zero. This means a zero-value [transfer](#), [mint](#) or [burn](#) of an uninitialized token will revert due to an invalid lookup, rather than with an explicit error message.

Consider skipping the balance update when the `value` is zero.

**Update:** Acknowledged, will resolve. The OpenZeppelin development team stated:

*The straightforward fix would be to guard with `if (value > 0)` or `if (_balances.member(id))`. However, due to a known Compact compiler limitation ([compact#226](#)), both branches of an `if/else` are evaluated unconditionally at the constraint level. As a result, branching around the `lookup` would still fail. We intend to keep the current implementation for now and address this once the compiler limitation is resolved.*

## L-02 Non-Existent and Zero Allowances Have Inconsistent Behavior

`FungibleToken` stores allowances in the nested ledger map `_allowances`. The public `allowance` circuit returns `0` when either the owner key or spender key is missing. However, `transferFrom` forwards to `_unsafeTransferFrom`, which unconditionally calls `_spendAllowance(fromAddress, spender, value)`.

`_spendAllowance` first asserts that both `_allowances.member(owner)` and the spender sub-map membership are true. As a result, `transferFrom(owner, to, 0)` reverts unless a prior approval created the relevant map entries, for example through `_approve`, even though the effective allowance is semantically `0`. This diverges from ERC-20-style expectations that a missing entry is equivalent to a zero allowance and can cause unexpected reverts in composed flows that legitimately compute a zero transfer amount.

Consider bypassing allowance spending when `value == 0`, or treating missing allowance entries as `0` within `_spendAllowance` so that it matches the semantics of `allowance`. Consider also adding tests for zero-value `transferFrom` calls without a pre-existing allowance entry.

**Update:** Acknowledged, will resolve. The OpenZeppelin development team stated:

The fix of bypassing `_spendAllowance` when `value == 0` is blocked by the same Compact compiler limitation ([compact#226](#)) – both `if/else` branches are evaluated unconditionally at the constraint level. We intend to keep the current implementation for now and address this once the compiler limitation is resolved.

## L-03 Token URIs Persist Across Burn and Remint

Each `NonFungibleToken` stores its own `URI`. However, this value is neither cleared during a [burn operation](#) nor reset during a [mint operation](#). As a result, when a token is burned, its obsolete URI remains in storage. If a new token with the same ID is later minted, it inherits the stale URI until it is [overwritten](#).

Consider clearing the stored URI when a token is burned, or otherwise ensuring that reminting a previously burned token ID cannot expose stale metadata.

**Update:** Resolved in [pull request #436](#).

## L-04 Misleading Documentation Regarding `DEFAULT_ADMIN_ROLE`

The `AccessControl` contract [documents](#) a mechanism to customize the `DEFAULT_ADMIN_ROLE`, which is used as the default administrator for other roles.

However, the contract does not use [the `DEFAULT\_ADMIN\_ROLE` in storage](#). Instead, when there is no explicit admin entry, `getRoleAdmin` returns a [hardcoded constant \(all zeros\)](#). Any customizations to the `DEFAULT_ADMIN_ROLE` ledger entry are non-functional and misleading.

Consider removing the `DEFAULT_ADMIN_ROLE` ledger entry and associated documentation.

**Update:** Resolved in [pull request #438](#).

## L-05 Owner Can Be Zero

The `Ownable` contract [claims to support experimental circuits](#) to grant ownership to contract addresses. However, it [authenticates against the left side of the owner](#), which corresponds to the [public key side](#). Therefore, the experimental circuits cannot be used.

Moreover, this discrepancy can be used to bypass the [non-zero owner validation](#) by setting `is_left` to `false` and a non-zero (irrelevant) contract address. This will [pass validation](#) but the actual owner would still be the public key, which could be set to zero.

Consider allowing the `assertOnlyOwner` circuit to support contract addresses.

**Update:** Resolved in [pull request #439](#).

## L-06 Code Simplifications

The `Uint128 _mul` function multiplies two 128-bit numbers by combining [all four combinations of 64-bit products](#). However, the implementation processes intermediate values and carries incrementally, while tracking their original source. For example it [first sums the low-half of the cross terms](#) and then later [adds the high-half of the lowest term](#). This adds unnecessary complexity.

Consider aligning the partial products to their respective positions and then processing all 64-bit words at each position in one operation. Here is a sample implementation:

```

circuit _mul(a: Uint128, b: Uint128): Uint256 {
  // Work with 64-bit words
  // Compute partial products (each is Uint<128> comprising 2 words within a 4-word result)
  const ll = toUint128(a.low * b.low); // 0      || 0      || ll.high || ll.low
  const hl = toUint128(a.high * b.low); // 0      || hl.high || hl.low  || 0
  const lh = toUint128(a.low * b.high); // 0      || lh.high || lh.low  || 0
  const hh = toUint128(a.high * b.high); // hh.high || hh.low  || 0      || 0

  // add the values at word 1. offset1.high is the carry, which is added to the next word
  const offset1 = toUint128(ll.high + hl.low + lh.low);
  // add the values at word 2. offset2.high is the carry, which is added to the next word
  const offset2 = toUint128(offset1.high + hl.high + lh.high + hh.low);
  // add the values at word 3. There should be no carry here since the result fits in 256 bits
  const offset3 = toUint128(offset2.high + hh.high);

  const resultLow = Uint128 { low: ll.low, high: offset1.low };
  const resultHigh = Uint128 { low: offset2.low, high: offset3.low };
}
```

```
assert(isZero(offset3.high), "Uint128: unexpected multiplication overflow");

return U256 { low: resultLow, high: resultHigh };
}
```

Additional simplifications were also identified:

- [These two lines](#) are redundant and could be removed.
- The `isApprovedForAll` circuit could replace the `if` statement with a single boolean statement (i.e. return whether all three conditions are satisfied).

Consider implementing these simplifications to improve code quality.

**Update:** Resolved in [pull request #313 of `midnight-apps` repository](#) and [pull request #440 of `compact-contracts` repository](#).

## L-07 Missing Admin Role Holder

The `ShieldedAccessControl` contract should assign at least one role holder (typically for the `DEFAULT_ADMIN_ROLE`) during initialization. Otherwise, the [access control mechanism](#) will prevent any role from ever being granted.

However, the intended mechanism to achieve this is unclear.

Consider updating the [initializer](#) to require a default admin address. Alternatively, consider calling the initializer and then the (unauthenticated) `_grantRole` circuit in the [example constructor documentation](#).

**Update:** Resolved in [pull request #445](#).

## L-08 Misleading Comments

Throughout the codebase, we have identified multiple instances of misleading comments as follows:

- The docstring above the `mint` circuit in `FungibleToken` contract mentions a non-existent `to` address.
- The `_approve` docstring [is missing the word "of"](#) in `NonFungibleToken` contract.
- The [error message](#) "NonFungibleToken: Nonexistent Token" does not match the check it's assert function is performing.

These inconsistencies can mislead readers and increase maintenance overhead. Consider updating those comments to avoid confusion over the codebase and improve readability.

**Update:** Resolved in [pull request #437](#).

# Notes & Additional Information

## N-01 Typographical Errors

Throughout the codebase, we identified several instances with typographical errors as follows:

- Incorrect spelling 'assertOnlyOnwer' in `ZOwnablePK` and `Ownable` contracts.
- Repetitive words at multiple instances: [\(1\)](#), [\(2\)](#), [\(3\)](#), and [\(4\)](#).
- [Module title](#) "Shielded AccessControl" should be updated to "ShieldedAccessControl".
- [Wrong proposition](#) "than" instead of "to" in `eq(Uint<128>)` docstring.
- Step numbering [skips Step 2](#) in `_div` circuit.
- [Missing space](#) in word "wit\_sqrtU128Locally" in `_sqrt` docstring.
- The [@title docstring](#) of `_toUint64` circuit names the wrong circuit

Consider updating those comments and documentations to improve code quality and consistency.

**Update:** Resolved in [pull request #446 of compact-contracts](#) repository and [pull request #312 of midnight-apps](#) repository.

# Conclusion

The audit identified 1 critical issue and multiple low & note severity issues. The critical issue stemmed from the incorrect assumption that `ownPublicKey` is a reliable source for authorization checks. However, `ownPublicKey` is a witness that can be set arbitrarily bypassing authorization checks and affecting majority of the contracts.

Addressing this critical issue may require substantial changes in the affected unshielded contracts. The code is heavily documented and of high quality.

The OpenZeppelin Contracts development team is appreciated for their active responsiveness and providing detailed response to our discussions. The team is commended for identifying a compiler level issue during one of those discussions.

# Appendix

## Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

### Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

### High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

### Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

### **Low Severity**

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

### **Notes & Additional Information Severity**

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.